Testing Evidence

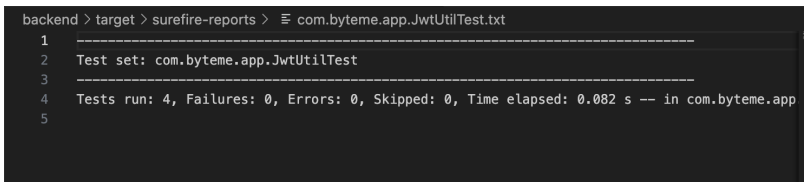
Testing Framework: JUnit 5, Mockito, Spring Boot Test (DataJpaTest), MockMvc

Security & Authentication Testing (JwtUtilTest)

What was done:

- Validated the generation and parsing of JSON Web Tokens (JWT).
- Tested the extraction of User ID, Email, and Role from encrypted payloads.
- Verified that expired tokens and malformed strings are correctly rejected.

Justification: Security is our top priority. By testing the JWT system, we make sure that only the right users can log in and access their own data. This stops "identity spoofing," where someone might try to pretend to be another user to steal information or cause trouble.



```
backend > target > surefire-reports > com.byteme.app.JwtUtilTest.txt
-------------------------------------------------------------------------------
Test set: com.byteme.app.JwtUtilTest
-------------------------------------------------------------------------------
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.082 s -- in com.byteme.app
```

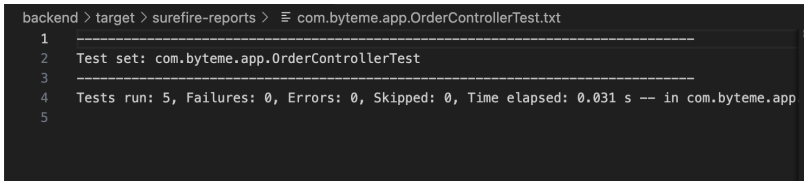
Figure 1: JwtUtilTest: Validation of Secure Token Lifecycle

Inventory & Transactional Logic (OrderControllerTest)

What was done:

- Simulated successful order creation and stock reservation.
- Tested edge cases, specifically preventing orders when stock is insufficient.
- Verified the full lifecycle of an order: `RESERVED` → `COLLECTED` or `RESERVED` → `CANCELLED`.

Justification: When dealing with food rescue, accurate numbers are vital. We don't want a charity to show up for food that isn't there. This testing fixes the "double-booking" problem at the source and ensures that if someone cancels an order, the food automatically goes back on the list for someone else to claim.



```
backend > target > surefire-reports > com.byteme.app.OrderControllerTest.txt
-------------------------------------------------------------------------------
Test set: com.byteme.app.OrderControllerTest
-------------------------------------------------------------------------------
Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.031 s -- in com.byteme.app
```

Figure 2: OrderController: Transactional Stock Management Evidence

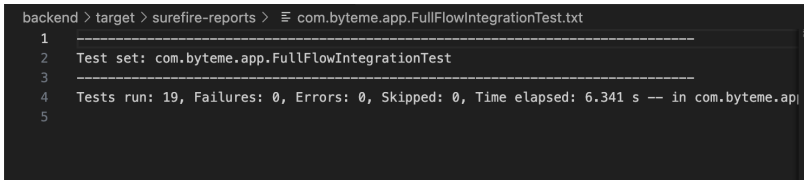


Figure 3: Full Flow Integration: Validating the path from Reservation to Collection

Automated CI/CD Pipeline Integration

What was done:

- Configured Railway's native CI/CD integration with the GitHub repository.
- Established an automated trigger that initiates the deployment pipeline upon every commit.
- Integrated the execution of the full JUnit test suite as a mandatory build step.

Justification: We use Railway and GitHub to check our code quality automatically every time we save our work. This "feedback loop" means the computer double-checks our tests for us. It helps catch human mistakes early and guarantees that the live app is always using a stable, working version.

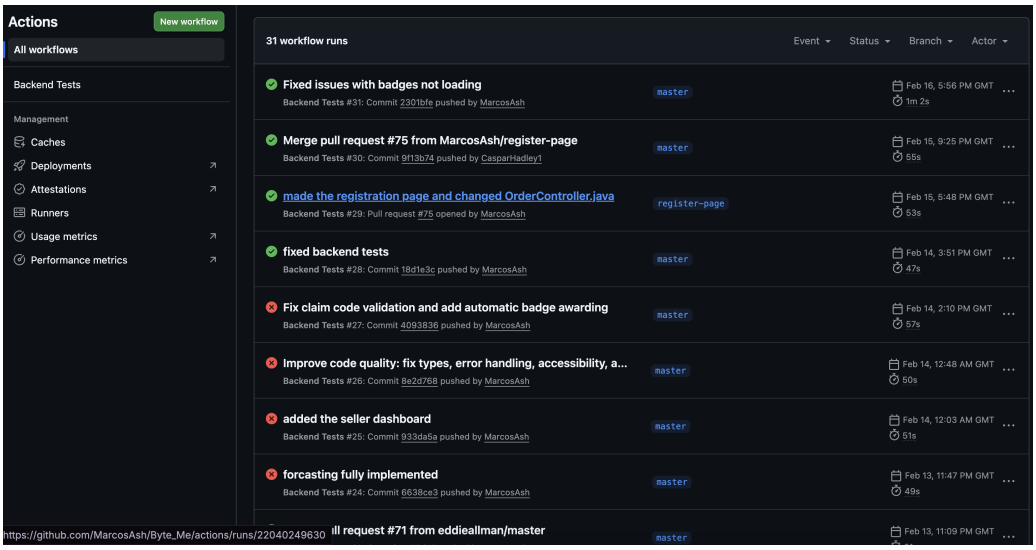


Figure 4: Railway CI/CD Pipeline: Automated Test Suite Execution Logs

Data Integrity & Persistence (Repository Tests)

What was done:

- Used `@DataJpaTest` to verify the schema constraints (Unique emails, Not Null names).
- Verified custom query methods, such as finding issues by Seller ID.
- Tested the immutability of `createdAt` timestamps using `updatable = false` validation.

Justification: The database is where all our important information lives. These tests make sure the "rules" of our data are followed—like making sure two people don't sign up with the same email. This prevents "dirty data" from getting into the system, which would otherwise cause the app to crash later on.

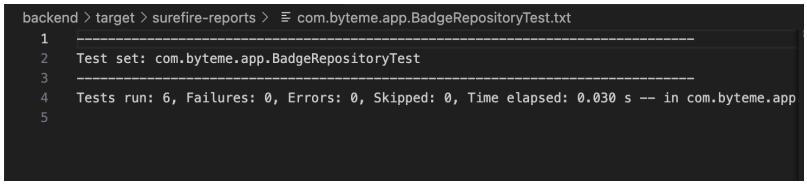


Figure 5: Repository Layer: Database Constraint and Query Validation

Gamification & Engagement (GamificationControllerTest)

What was done:

- Mocked the retrieval of user streaks (Current and Best).
- Verified the logic for awarding badges like `STREAK_4` and `CO2_SAVER`.
- Tested DTO mapping to ensure the frontend receives correctly formatted JSON.

Justification: We use a streak system to encourage users to keep donating and rescuing food. Testing this ensures the game is "fair"—streaks should only go up when a user actually completes a task. This keeps users engaged and ensures they get the badges they've actually earned.

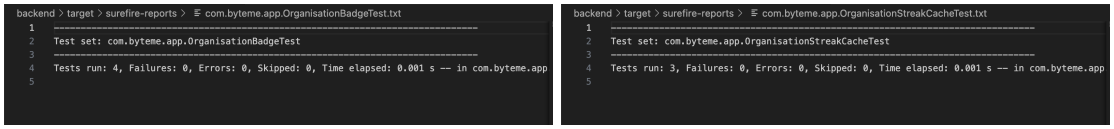


Figure 6: Gamification: Badge Award Logic and Streak Persistence

Issue Reporting & Conflict Resolution (IssueControllerTest)

What was done:

- Verified that Organisations can report issues with orders.
- Tested the "Resolve" flow and automatic generation of resolution timestamps.

Justification: Trust is key to a successful marketplace. If there is a problem with an order, we need a reliable way to track and fix it. These tests ensure that every complaint is recorded and that there is a clear "paper trail" showing when and how a problem was solved.

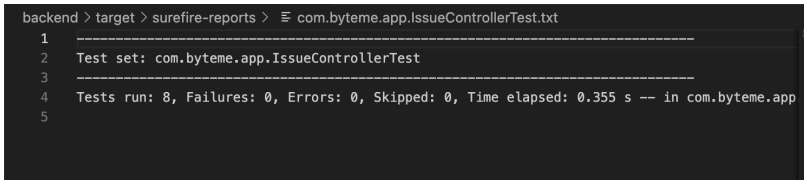


Figure 7: IssueController: Audit Trail and Dispute Resolution Testing

Domain Model Validation (Entity Tests)

What was done:

- Tested Pojos like `UserAccount`, `Seller`, and `Organisation`.
- Verified that default values are assigned upon object instantiation.

Justification: Entities are the basic building blocks of our app. We test them to make sure the "foundation" is solid. If the simple things (like setting a user's name) don't work, the complex parts of the app won't work either. This keeps our data consistent from the start.

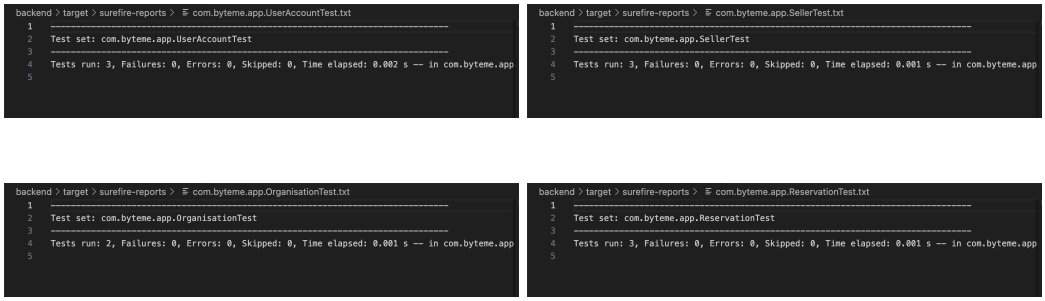


Figure 8: Entity Validation: Ensuring structural integrity of core Domain Objects